

4 — Models: architecture, normalisation, evaluation

1 Network

Aim

One network, three feature arms:

- **PH arm:** persistence images, one per (filtration, homology dimension)
- **PersLay arm:** the same diagrams, vectorised by the network instead of rasterised
- **Classical arm:** sampled summary functions, L, F, G, J

Two tasks:

- **Classify:** 5-way family (Poisson, Thomas, nested, Matérn II, LGCP)
- **Params:** the family's own parameters, one model per family

Everything after the feature step is shared, so the arms are comparable by construction.

Inputs

- **Images:** 64×64 (or just 64 for Rips and alpha H_0)
- **Diagrams** (PersLay): padded to a fixed length, columns (birth, persistence, mass)
- **Curves:** 512 samples each
- **Covariate:** $\log n$, the realised count, one scalar

Architecture (PHNet)

- **Encoder**, one per input key, 1-D or 2-D following the input's rank:

[conv₃ 32 → ReLU → avgpool₂ → drop] → [conv₃ 64 → ReLU → avgpool₂ → drop]
→ [conv₃ 128 → ReLU]
→ adaptive avgpool(4)
→ flatten
→ Linear
→ ReLU,

embedding dimension 64. Padding 1, stride 1, so the convolutions keep resolution and only the pools shrink it: $64^2 \rightarrow 32^2 \rightarrow 16^2 \rightarrow 4^2$ in 2-D, $512 \rightarrow \dots \rightarrow 4$ for a curve

- **Head:** concat(embeddings, $\log n$) → MLP [64, 32], ReLU, dropout 0.1 → 5 logits, or p targets

PersLay encoder

A diagram is a *set*, so any vectorisation of it is permutation invariant, and PersLay (Carrière et al. 2020) writes every such map in one form,

$$V(D) = \text{op}(\{w(p) \varphi(p) : p \in D\}),$$

with a point transformation φ , a scalar weight w and a permutation-invariant pooling op. The persistence image is the frozen member of that family: φ a Gaussian on a fixed grid of centres, $w(p)$ the linear weight (the persistence), op the sum. This arm keeps that shape and *learns* it:

- φ : $Q = 64$ Gaussians with learned centres and bandwidths, in the standardised (birth, persistence) plane the imager also works in; centres initialised spread over $[-2, 2]^2$, bandwidth $4/\sqrt{Q}$
- w : $\text{softplus}(\text{MLP}_{2 \rightarrow 16 \rightarrow 1}(p))$, non-negative because a weight is a mass, multiplied by the point's own mass ($1/n$ under density scaling), so the n -scaling of the image arm carries over unchanged
- op: sum (max and mean are available), then dropout, Linear, ReLU to the same embedding dimension 64 the image encoder produces

Capacity is fitted like the imager's box: it holds every point of 99% of the *training* diagrams, capped at 1024, and a longer diagram keeps its most persistent points — the ones the image's linear weight would have made brightest. Padded rows carry mass exactly 0, so padding is invisible to the pooling.

How inputs combine

- H_0 and H_1 : separate encoders, embeddings concatenated
- Several filtrations (multi- k): one *shared-weight* encoder applied to each in turn, embeddings concatenated
- Curves on one grid: stacked as channels of the first convolution, since index i is then the same r in every channel and the convolution can compare them there
- Curves on different grids: one encoder each, meeting only after pooling

Design choices

- Pool to 4 instead of global. A conv stack under a global average is translation-invariant for *every* weight setting; in a persistence image position *is* the measurement. One blob at two positions: relative embedding distance 0.0002 under a global pool, 0.14 at pool 4
- Channel-wise dropout (Dropout1d/2d, $p = 0.2$): neighbouring pixels are correlated, so element-wise dropout regularises little
- No normalisation layers anywhere (no BatchNorm, no LayerNorm). All normalisation is on the input and target side, where it is explicit and fitted on train rows only

Parameter counts (classification head). Capacity is a property of the input layout, not something matched across arms:

Input	encoder(s)	head	total
DTM H_0+H_1 (2-D + 2-D)	$2 \times 223,808$	10,565	458,181
Rips/alpha H_0+H_1 (1-D + 2-D)	$63,872 + 223,808$	10,565	298,245
Rips/alpha H_0 (1-D)	63,872	6,469	70,341
L, F, G, J stacked (1-D, 4 channels)	64,160	6,469	70,629
L, F, G, J separate	$4 \times 63,872$	18,757	274,245
PersLay H_0+H_1 ($Q = 64$)	$2 \times 4,417$	10,565	19,399
PersLay H_0 ($Q = 64$)	4,417	6,469	10,886

2 Normalisation of covariates and targets

- $\log n$, z -scored on train
- Targets: $\log$ then z -score per target. Loss is MSE on the standardised log targets, so each target is weighted equally whatever its actual scale
- Predictions are inverted ($\cdot \text{sd} + \text{mean}$, $\exp$) and saved in original parameter units

- Per family: Poisson $\bar{n}$; Thomas κ, μ, σ ; nested $\kappa, \mu_1, \mu_2, \sigma_1, \sigma_2$; Matérn R, λ_p ; LGCP $\bar{n}, \sigma^2, s$ ($\bar{n}$ in place of $\mu_{\log}$, the only target that can be negative)

3 Evaluation

Split

- By θ index, not by pattern: train $\theta < 7000$, val $7000 \leq \theta < 8000$, test $8000 \leq \theta < 10\,000$
- Replicates of a θ fall in the same split, so no parameter vector is seen in both train and test
- θ_i is drawn from its own stream, so indices are i.i.d. prior draws and a contiguous block is a random split
- The same split for every family, task, arm and seed

	train	val	test
θ per family	7000	1000	2000
Patterns, classify (5 families)	70 000	10 000	20 000
Patterns, params (1 family)	14 000	2 000	4 000

Training

- AdamW, lr 10^{-3} , weight decay $3 \cdot 10^{-4}$, batch 128
- Loss: cross-entropy (classify), MSE on standardised log targets (params)
- Up to 200 epochs; early stopping on val loss, patience 30; best-val weights restored
- Val selects the stopping epoch and nothing else
- The seed sets network init, minibatch order and dropout. Features and split are identical across seeds, so seeds measure optimisation variance, not data variance

Metrics

- Every test pattern’s prediction is saved with its `case_id` and joined to the manifest for $(\bar{n}, \tilde{\delta})$ afterwards, so the binning can change without retraining
- **Classify**: accuracy, and NLL of the true family. Restricted to one family, accuracy is that family’s recall
- **Params**: RMSE in log units, per target and pooled over targets
- **Cells**: 8 $\log\tilde{\delta}$ bins on $[0.1, 4] \times 3 \log\bar{n}$ bins on $[100, 800]$, plus every marginal. Edges come from the prior and are shared by every family and method; cells with fewer than 10 test θ are skipped
- Poisson has $\tilde{\delta} = 0$ and so falls in no $\tilde{\delta}$ bin: it appears in the marginal and $\bar{n}$ rows only. A $\tilde{\delta}$ cell of the classification task therefore holds four families, and chance in it is 0.25